/

Claude finished the response

1793219890419_LlamaVascularExtractor.java

java

1793219890427_MainVascular.java

java

1793219890428_NullToNoInformationDeserializer.java

java

1793219890429_ValueLocator.java

java

1793219890430_VascularDbImporter.java

java

1793219890430_VascularModels.java

java

1793219890431_VascularRunner.java

java

1793219890432_VascularThroughputHarness.java

java

1793219890433_VascularTsvWriter.java

java

I am building a Java application using the Eclipse IDE. The application is designed to extract arterial flow velocities, percentage of stenosis, and internal to common carotid artery ratios from radiology reports on vascular flow in the supplying vessels of the head. The reports are from ultrasound-generated reports in the VA on Veterans with suspected vascular disease.

We are planning to use a quantized large language model, Gemma 4, that is hosted by a llamacpp application running locally. Currently, we are running the application on a Intel Silver 4310 CPU with 256 Gb of RAM and a Nvidia A2 GPU. We are starting the llamacpp application with the following command:

start "" "P:\ORD_Nguyen_202206062D\Glenn\LlamaAppInstall_261008\llama-b11514-bin-win-cuda-12.4-x64\llama-server.exe" --host 127.0.0.1 --port 8080 --model "O:\OMHSP_CONDUIT\NLP\Glenn\LLMs\Models\gemma-4-E4B-it-IQ4_NL\gemma-4-E4B-it-IQ4_NL.gguf" --n-gpu-layers 999 --ctx-size 49152 --parallel 4 --cont-batching --batch-size 2048 --ubatch-size 512 --flash-attn on --threads 12 --load-mode mlock

I have pasted in the Java code we are using in this prompt, and I would like you to review that code. Then I will ask you to make changes to the code to address some additional needs.

Show more

Read 9 files

Read 9 files

Let me check the uploads directory in the sandbox.

Ran 4 commands

Ran 4 commands

I read all nine files. The architecture is sound: a system/user prompt split for KV-cache reuse, a JSON schema constraint, a truncation-aware retry, per-document failure isolation, and a locator that finds offsets in Java instead of trusting the model. Below are the problems I'd fix, roughly in priority order.

Accuracy risks

1. **Vertebral and subclavian vessels have no place in the schema.** Carotid duplex reports almost always include vertebral artery velocities and direction.
   The vessel enum only allows ICA, ECA and CCA, and the prompt never says to skip other vessels. The model will either mislabel a vertebral value as ICA or CCA, or drop it unpredictably. Either add VERTEBRAL or Other to the enum, or add an explicit "ignore all other vessels" rule. Also, vessel_location has no "bulb" or "bifurcation" option, which reports use often.
2. **Forced defaults hide missing data.** If the report doesn't say PSV or EDV, the model must write PSV. If it doesn't state flow direction, the model must write anterograde.

Downstream, an unstated value is indistinguishable from a stated one. I'd add Unspecified to both enums, in the schema, prompt, repair prompt and validate(). You can then decide in analysis how to treat those rows.

3. **Reports often state the same finding twice.** Stenosis commonly appears in both the body and the Impression. Nothing deduplicates these, so you may get double counts. Decide on a policy, such as keeping both and tagging the section, or collapsing duplicates.
4. **Units are dropped silently.** If a report ever gives m/s, the value is stored without a flag. This is low risk, but a sanity check on plausible ranges would catch it.

Robustness bugs

1. **Files.readString(path) assumes UTF-8.** VA text exports containing Windows-1252 characters (degree signs, "µ", smart quotes) throw MalformedInputException, and the document fails. Read the bytes and decode with a fallback charset. Offsets then stay consistent with the text you pass to the model.
2. **There is no context-fit guard.** Each slot holds 12,288 tokens, and you reserve 7,168 for output. Any prompt over about 5K tokens (a long multi-study report) overflows. The retry at 8,192 would then fail the same way. Count tokens (llama-server has /tokenize, or estimate from character count) and lower max_tokens for long documents. Flag documents that can't fit.
3. **The HTTP request has no timeout.** If the server stalls, that worker thread hangs forever, and analyzeDocumentsParallel never returns. Add .timeout(...) to the request and a connect timeout on the client. I'd also pin HttpClient.Version.HTTP_1_1, since llama.cpp doesn't speak HTTP/2 and the default client tries an h2c upgrade first.
4. **Runaway generation is a plausible cause of your 4,300–4,800 token outputs.** Greedy decoding with a repeat penalty of 1.0 is prone to repetition loops in small models. A typical report needs about 1,000 tokens (your example with 13 velocities is roughly that). Check a few of those long outputs for repeated entries. If you find loops, add duplicate detection and consider streaming so you can abort early. Also check whether Gemma 4's chat template is emitting thinking or reasoning tokens. If so, disabling them would cut generation time. I can't verify that for this build.
5. **The locator's cursor assumes document order.** It picks the first candidate after the previous value's start offset. If the model groups output by vessel while the

report is organized by side, repeated excerpts like CCA: PSV 82 cm/s can resolve to the wrong occurrence. Tracking a set of already-claimed offsets, instead of only a monotonic cursor, would be more robust.

6. **Resume and rerun are weak.** Output is append-only, so rerunning duplicates records. The _failed.txt list is written, but nothing reads it back. Add a "process only these paths" option and skip documents already present in the output.
7. **Zero-finding documents leave no trace in the TSVs.** You can't tell "processed, nothing found" from "failed" without the log. A fourth documents.tsv with status, counts and match statistics would fix that.

Smaller issues

- VascularRunner runs the whole extraction on the Swing event-dispatch thread. An uncaught exception, such as a bad argument to RunType.valueOf, skips System.exit(0) and leaves the JVM hanging. It also uses scanner.next(), which breaks on paths containing spaces.
- getOutputFile() closes System.in through try-with-resources on Scanner. This works today but will break any later console input.
- estimateRemainingProcessingTime divides by zero if elapsed time is 0 ms.
- The DB importer doesn't store source_text, offsets or match method, so the database is less informative than the TSVs. It also has no duplicate protection on reimport, and it calls System.exit inside a library method.
- Several comments and usage messages still refer to the …Revised class names.
- ExecutorService as an AutoCloseable needs Java 19 or later, so check the Eclipse compliance level.
- In the throughput harness, a 20-document sample with 4 threads is skewed by tail effects, since the last documents run with idle slots. The warm-up document is also in the timed sample. Capturing llama.cpp's timings (tokens per second) from each response would give you cleaner numbers.

Server command

Everything in the launch line is sensible for a 16 GB A2 with all layers on the GPU. --mlock matters little when weights live in VRAM. I can't verify --load-mode mlock against build b11514. The older spelling is plain --mlock, so confirm with llama-server --help that the flag is actually taking effect.

Which changes would you like me to make first?

Let's go through these issues one at a time.

Regarding "Accuracy risks"

1. Please add an explicit "ignore all other vessels" rule, and include "bifurcation" and bulb as vessel location options.
2. Please add "Unspecified" to the PSV or EDV and the anterograde or retrograde options.
3. Continue to keep duplicates. Do not collapse them.
4. Please add the units as a separate item to extract within flow_velocities. The options for units should be "cm/s", "m/s", "Other", or "Unspecified."

Thought process

Thought process

I'll copy the five affected files into my working directory and apply the edits there. The originals stay untouched.

Ran 6 commands, read a file

Ran 6 commands, read a file

I made all four changes in five files: MainVascular, LlamaVascularExtractor, VascularModels, VascularTsvWriter and VascularDbImporter. I couldn't compile them here because the sandbox has no javac. I checked each change by diffing against your originals, so please run a build in Eclipse. The other four files (ValueLocator, VascularRunner, VascularThroughputHarness, NullToNoInformationDeserializer) are unchanged.

1. Other vessels and new locations

- **Other vessels:** The system prompt now says to extract only ICA, ECA and CCA. It tells the model to ignore the vertebral, subclavian, innominate, basilar and intracranial arteries, and not to relabel their values as ICA, ECA or CCA.
- **Few-shot example:** I added a vertebral artery line to the example report. It has no matching entry in the example output, so the model sees the ignore rule applied.

- **Locations:** vessel_location now allows bulb and bifurcation. I updated this in the prompt, the JSON schema, the repair prompt, and a new location check in validate().

2. "Unspecified" for velocity type and flow direction

- **Enums:** velocity_type is now PSV, EDV or Unspecified. flow_direction is now anterograde, retrograde or Unspecified.
- **Prompt rules:** The old "default to PSV" and "default to anterograde" rules are gone. Retrograde is used only when the report says so. Anterograde is now used only when the report explicitly says antegrade or anterograde. Everything else is Unspecified.
- **Example output:** Every flow direction in it changed from anterograde to Unspecified, because that example report never states direction. Leaving it unchanged would have taught the model to keep guessing.
- **NoMeasurableFlow entries:** They now use the stated velocity type, or Unspecified, instead of forcing PSV.

3. Duplicates

- Nothing changed, and nothing in the pipeline removes duplicates. The prompt doesn't ask the model to collapse them. I can add an explicit rule to report every occurrence separately if you see the model skipping repeats.

4. Units

- **New field:** flow_velocities has a new units field with the options cm/s, m/s, Other and Unspecified. It sits right after value in the prompt schema, JSON schema, repair prompt, both model records, validate() and the console summary.
- **Prompt rule:** The prompt tells the model to record the unit as stated, never convert it, and keep it out of value. Every velocity in the example output now carries "units":"cm/s".
- **TSV:** units is a new column after value in the flow velocities file.
- **Database:** The importer now writes Units as the eighth parameter of the velocity insert.

Before you run anything, note two things.

- **Database column:** ggLLM.FlowVelocity.Units must exist as a nullable column, such as NVARCHAR(50). I updated the header comment in the importer with the ALTER TABLE note.
- **Mixed TSV files:** If you append to a flow velocities file made by the old version, its header and rows won't match the new column layout. Start a new output file for the next run.

Which accuracy issue would you like next, or should I move on to the robustness bugs?

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\MainVascular.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\LlamaVascularExtractor.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularModels.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularTsvWriter.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularDbImporter.java

Code

Thought process

Thought process

Let me check the files for Windows-specific assumptions first, so my advice on code changes is accurate.

Ran a command

Ran a command

The Java code is mostly portable. The server manager uses ProcessHandle and destroy(), with no Windows-only calls, and the file I/O sets UTF-8 explicitly. The Windows-specific parts are the hard-coded paths and the server flags. I haven't run any of this on a Mac, so the speed and memory figures are estimates, and the Mac changes are untested.

What changes with this hardware

- **Unified memory:** the GPU and CPU share the 96 GB, so there is no separate 15 GB VRAM ceiling. macOS lets the GPU use roughly 75% of RAM by default, which is about 72 GB. The server log reports the exact limit when it loads the model.
- **Much more headroom:** the model is about 5 GB, so you can raise the context, the slot count, or both.
- **Speed:** my rough expectation is that the M2 Max GPU has more compute than your A2, and about twice its memory bandwidth (about 400 GB/s against 200 GB/s). I'd expect it to be at least as fast as the Xeon + A2 machine, but I haven't measured this. Use the throughput harness to find out.

Server

Executable: download the llama.cpp **macOS arm64** build. Metal is enabled by default. Do not use an x64 build, which would run under Rosetta without Metal.

First run:

- Make the binary executable: chmod +x llama-server.
- Clear Gatekeeper's quarantine flag: xattr -dr com.apple.quarantine <folder>.

Setting	Current	Suggested
--n-gpu-layers	999	Keep. It offloads everything to Metal.
--threads	12	About 8, the number of performance cores. With full GPU offload the CPU does little, so this matters less than before.
--load-mode mlock	on	**Remove it.** Weights live in unified memory already, and mlock can hit macOS locked-memory limits.
--flash-attn on	on	Keep. It is supported on Metal.

Setting	Current	Suggested
--batch-size / --ubatch-size	2048 / 512	Keep.
--cont-batching	on	Keep. It matters with several slots.
--ctx-size / --parallel	49152 / 4	Start at **98304 / 8**, which is still 12,288 tokens per slot. Then sweep (see below).

Slots and context:

- The context cost is the KV cache, whose size I can't compute for Gemma 4 precisely. The server startup log prints it, so check it against free memory.
- Leave at least 20 to 25 GB free for macOS, Java and Eclipse.
- Decode speed is limited by memory bandwidth, so more slots help only up to a point. Use VascularThroughputHarness with a sweep of {2, 4, 6, 8} and pick the best docs/min.
- Alternatively, use fewer, larger slots, for example 4 slots × 24,576 tokens. That admits longer reports in the first pass and may make the second pass unnecessary.

Model quantization (optional):

- Memory allows a higher-quality quantization such as Q8_0, about 8 GB, instead of IQ4_NL. Accuracy may improve a little.
- Generation is bandwidth-bound, so a Q8_0 file should run roughly 1.5 to 2 times slower.
- Larger Gemma 4 variants (12B, 26B) also fit, but would be slower still.
- Any of these changes the results relative to your production configuration, so compare accuracy on a labeled sample before adopting one.

Monitoring: nvidia-smi doesn't exist on a Mac. Use Activity Monitor's Memory Pressure and GPU History, or sudo powermetrics --samplers gpu_power.

Long runs: prevent sleep, for example by launching Eclipse's run with caffeinate -i, or by setting Energy Saver to never sleep.

Code changes (all small)

1. **LlamaServerConfig.vaDefaults():** change the executable path (no .exe), the model path, ctxSize, parallel and threads, and remove the --load-mode mlock extra argument.
2. **MainVascular:** change CORPUS_DIRECTORY_PATH, DEFAULT_PARALLEL (to 8, or whatever the sweep chooses) and SECOND_PASS_PARALLEL (1 or 2).
3. **VascularThroughputHarness:** change SWEEP_VALUES. SLOT_SIZE can stay at 12288.
4. **Paths on macOS:** use POSIX paths such as /Users/<you>/…. Java does not expand ~, so write the full path.
5. **LlamaServerManager:** only its "end llama-server.exe" hint message mentions Windows. Optionally make it generic.
6. **Java:** install an **arm64** JDK (19 or later) and the arm64 Eclipse.

My recommendation: you now have three environments (the Xeon + A2 machine, the small PC and the Mac). Instead of editing constants each time, I'd move the environment-specific values into one external properties file selected at startup:

- executable path
- model path
- ctx-size
- parallel
- threads
- extra flags
- corpus directory

That would be a change to LlamaServerConfig and MainVascular. I can do it if you'd like.

Prompt

No change needed. The prompt isn't hardware-dependent,
and ContextBudget recalculates the longest admitted report from it and the slot size. Larger slots will simply admit longer reports. Keep DISABLE_THINKING = true, and watch the end-of-pass usage summary for reasoning output, as before.

If you try a larger model or a higher-quality quantization, test with your current prompt first. Revisit the prompt only if you see systematic errors, such as extra vessels or non-unique excerpts.

Yes, please move the environment-specific values into one external properties file selected at startup.

Thought process

Thought process

I'll start by reading the current config class and the places that use these values.

Ran 2 commands

Ran 2 commands

I'll write the two new pieces first: the settings loader and the updated LlamaServerConfig, which no longer has the hard-coded vaDefaults().

Ran a command

Ran a command

I wrote package core; from memory, so I'm checking it against the original files.

Ran a command

Ran a command

The package is core, so that guess was right. Next is AppSettings.

Ran a command

Ran a command

The files compile. Next I'm making the BOM check explicit, then writing the three sample files and testing the parser.

Ran a command

Ran a command

The parser works. The Windows paths look odd above only because Linux doesn't treat P:\... as absolute, and they're absolute on Windows. Next I'm editing the remaining files.

Ran 5 commands, read a file

Ran 5 commands, read a file

All the environment-specific values now come from one external settings file. The same build can run on the Xeon and A2 machine, the small PC and the Mac Studio. I compiled and
tested AppSettings, LlamaServerConfig and LlamaServerManager. MainVascular, LlamaVascularExtractor, VascularThroughputHarness and VascularRunner are not compiled, because the sandbox only has Java 11 and no Jackson. Please build in Eclipse and tell me about any errors.

Choosing the file. The first match wins:

1. --config=PATH as a program argument. VascularRunner removes it before reading RunType.
2. -Dvascular.config=PATH as a VM argument in the Eclipse run configuration.
3. The VASCULAR_CONFIG environment variable.
4. A prompt at startup. Pressing Enter uses built-in defaults that reproduce the old hard-coded VA Xeon and A2 values, so nothing breaks if you set nothing.

The chosen file and the full resolved llama-server command are printed at startup, so each log shows which machine profile ran.

What's in the file:

- **Server:** executable, model, host, port, GPU layers, ctx-size, batch, ubatch, flash-attn, threads, cont-batching, extra args and the startup timeout.
- **Run:** default parallel, whether to run a second pass, second-pass parallel and model name.
- **Corpus:** directory and extension.
- **HTTP:** request and connect timeouts. These are now instance settings on the extractor, so the CPU-only PC can use a 60-minute request timeout.
- **Harness:** sweep values, slot size, sample size and warm-up.

Parsing rules:

- **Backslashes:** paths are not escape-processed, so you can paste P:\ORD... as-is. I wrote a custom parser because java.util.Properties would have corrupted those paths. Forward slashes also work.

- **Relative paths:** a relative path resolves against the settings file's folder.
- **Home folder:** ~ means your home folder.
- **Threads:** server.threads = 0 omits --threads.
- **Errors:** unknown keys give a warning. All bad or missing values are reported together in one error.

Sample files. vascular-va-xeon-a2.properties holds your current values. vascular-small-pc-cpu.properties and vascular-mac-studio-m2max.properties hold the settings from my earlier advice, with placeholder paths you need to edit. The small-PC profile uses 2 slots of 8192 tokens, 6 threads, a 60-minute request timeout and a 30-minute startup timeout. The Mac profile uses Metal with all layers offloaded, 4 slots of 12288 tokens and 8 threads.

LlamaServerConfig.vaDefaults() is gone. The prompt, schema and MAX_TOKENS stay in code because they belong to the task and model rather than the machine. The first-pass slot size now comes from the file's ctx-size. The Number of parallel server slots prompt shows that value instead of the old hard-coded 49152.

Files are in C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs:

- AppSettings.java
- LlamaServerConfig.java
- LlamaServerManager.java
- LlamaVascularExtractor.java
- MainVascular.java
- VascularThroughputHarness.java
- VascularRunner.java
- vascular-va-xeon-a2.properties
- vascular-small-pc-cpu.properties
- vascular-mac-studio-m2max.properties

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\AppSettings.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\LlamaServerConfig.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\LlamaServerManager.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\LlamaVascularExtractor.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\MainVascular.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularThroughputHarness.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\VascularRunner.java

Code

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\vascular-va-xeon-a2.properties

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\vascular-small-pc-cpu.properties

C:\Users\VHATVHGOBBEG\AppData\Local\Claude-3p\local-agent-mode-sessions\7ca07dd0\00000000\local_af1e8fc9-ab5e-46b3-a4ee-ea88e2f0d2b4\outputs\vascular-mac-studio-m2max.properties